
TECHNICAL DOCUMENTATION PACK · DOCUMENT 03

LAAM

Infrastructure, Security & Operations

How the conversational AI work assistant is built, deployed, secured and operated today, and what is planned before it runs in production.



VERSION
1.0

DATE
2026-09-24

CODE BASELINE
LAAM@7eb0724

Contents

1 Deployment topology

Reference stack · host-native services · ingress · external scheduler tick

2 Runtime & build

Standalone Next.js image · Node 24 · Drizzle migrations · health check

3 Configuration

Environment variable categories · secrets handling

4 Scalability model & current constraints

Single-process assumptions · target production topology (proposed)

5 Security architecture

Authentication · authorization · isolation · credential encryption · connector OAuth · SSRF · write-confirm gate · code-execution ban · headers · MCP server · controls table

6 Data protection & AI data flow

What leaves the premises, to whom, and when · fully local configuration

7 Operations

Observability · cost tracking · backups · updates & migrations · health checks

8 Hardening items before production

Status vocabulary used in this document

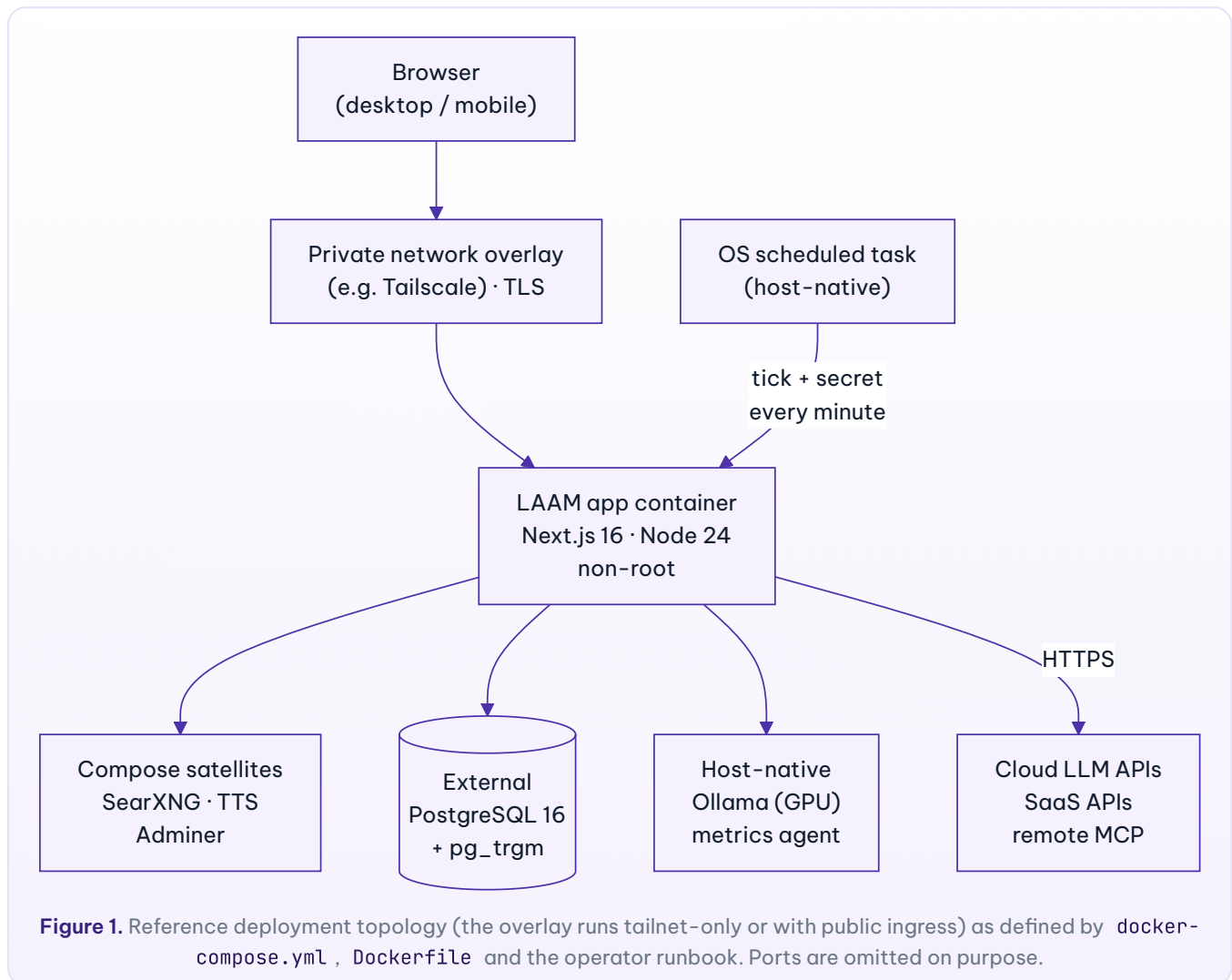
LIVE implemented, wired and exercised with real data in the self-hosted stack · **BUILT** implemented and wired, not yet exercised with production data · **PLANNED** roadmap, flag-off or not implemented.

Deployment status: pre-production

LAAM runs today as a self-hosted Docker Compose stack for development, internal use and demonstrations. There is no production environment yet (the repository calls the internal self-hosted deployment its "production" stack; it serves no customers), and this document makes no claims of production customers, SLAs, uptime or measured scale. Section 4.3 describes a **proposed** production topology, and Section 8 lists the hardening work planned before a production launch.

1 Deployment topology

LAAM is a single Next.js 16 application backed by PostgreSQL 16. Around it sit three small containers (web search, text-to-speech and a database admin UI), two host-native services that need direct access to the GPU and the host, and one external scheduler tick.



1.1 Components

Component	Runs as	Role	Status
LAAM app	Compose service, non-root container	All UI pages and 64 API route files. It hosts the chat and voice agent loop, the workflow engine, connectors, the MCP client and the MCP server. The container publishes one HTTP port to the host.	LIVE
PostgreSQL 16	External shared instance, reached over a Docker network	The system of record: 22 tables with <code>pg_trgm</code> trigram search. LAAM no longer runs its own database container, and stopping the stack leaves the database and its data untouched.	LIVE
SearXNG	Compose service, bound to loopback	Self-hosted metasearch behind the <code>web_search</code> tool. The app does not depend on it at start-up, so web search degrades gracefully when it is down.	LIVE

Component	Runs as	Role	Status
VieNeu TTS	Compose service (Python/FastAPI), bound to loopback, capped at 3 CPUs	Streaming neural text-to-speech for the voice surface (Vietnamese and English voices). The CPU and thread caps are set together. Measured on the reference host, this runs 3.58× real time without starving the browser.	LIVE
Adminer	Compose service	Web UI for database administration by the operator. Its port is published on the host (see Section 8, item 7).	LIVE
Ollama	Host-native (needs the GPU)	Local LLM runtime: the \$0, on-premises model path. The container reaches it through the Docker host gateway.	LIVE
Host metrics agent	Host-native, zero-dependency Node script	Samples CPU, RAM and GPU (via <code>nvidia-smi</code>) about once a second for the voice command-centre's system panel. Read through an app proxy that fails soft; an optional bearer token is supported.	BUILT
Workflow tick	OS scheduled task	Sends a POST to the tick endpoint every minute with a shared secret. This claims due schedules and resumes interrupted runs. There is no in-process cron.	LIVE
Monitoring collector	Zero-dependency Node script on each monitored workstation	Pushes parsed coding-agent session metadata and short text snippets (never full transcripts) to the ingest endpoint using a per-machine collector token. It retries with jittered backoff.	BUILT

1.2 Ingress and network exposure

- **Ingress** is designed to go through a private network overlay (for example Tailscale). The overlay terminates TLS and offers two modes: *tailnet-only*, reachable from enrolled devices only, for development, and *public ingress* for demonstrations. The app trusts the forwarded host (`trustHost`), as a self-hosted deployment behind a proxy requires. The app and database-admin ports are also published on the host, so exposure on the local network is governed by host firewalling (Section 8, item 7).
- **Internal dependencies** (SearXNG, TTS) are published on the loopback interface only. The app reaches them by Compose service name.
- **Outbound** traffic from the app goes to cloud LLM providers, connector SaaS APIs, user-configured remote MCP servers and keyless public geo/weather services. Every one of these depends on configuration; see Section 6.

Development mode

Developers run the same code with `npm run dev` (Turbopack) against the same database schema. The Docker image is used for internal and demonstration hosting.

2 Runtime & build

Node 24

ALPINE RUNTIME
(ACTIVE LTS)

3-stage

DOCKER BUILD · DEPS →
BUILD → RUN

19

DRIZZLE SQL
MIGRATIONS
(JOURNALED)

non-root

CONTAINER USER

2.1 Application runtime

Layer	Choice
Framework	Next.js 16.2 App Router, <code>output: "standalone"</code> . The image ships a self-contained <code>server.js</code> and needs neither the full <code>node_modules</code> nor <code>next start</code> .
UI · validation	React 19.2, TypeScript 6 (strict), Tailwind CSS 4; <code>zod</code> 4 at request boundaries
Auth	Auth.js v5 beta (<code>next-auth</code> 5.0.0-beta.31) with the Drizzle adapter; <code>bcryptjs</code> for password hashing
Data access	<code>drizzle-orm</code> 0.45 over <code>node-postgres</code> (<code>pg</code> , kept as a server-external package so it is not bundled)
AI / MCP	Anthropic SDK for Claude; OpenAI-compatible providers over plain <code>fetch</code> (no SDK); <code>@modelcontextprotocol/sdk</code> for the MCP client (Streamable HTTP with SSE fallback); LAAM's own MCP server is a hand-written JSON-RPC handler over Streamable HTTP
System binaries in image	<code>poppler-utils</code> (server-side PDF text and rasterisation), <code>tesseract-ocr</code> with English, Vietnamese and Simplified Chinese data (OCR), <code>unzip</code> (DOCX extraction)

2.2 Container build

- deps:** `npm ci` from the lockfile on `node:24-alpine`. The Node/npm major is pinned to match development hosts, to avoid lockfile-dialect drift.
- builder:** `next build` with `NODE_ENV=production` and telemetry disabled. Static assets are copied into the standalone output.
- runner:** a minimal image. The OCR and PDF packages are installed as root, then the image switches to the built-in unprivileged `node` user before the app is copied in. The entry point is `node server.js`.

The image is built by the operator (`docker build`) and tagged locally. There is no container registry or CI/CD pipeline yet (Section 8).

2.3 Database migrations

- The schema lives in one TypeScript file (`src/db/schema.ts`). SQL migrations are generated with `drizzle-kit generate`, committed, and applied with `drizzle-kit migrate`. The journal is the source of truth for which migrations have run.
- Policy:** real data is changed through migrations only. `db:push` is never used on a real database. Two migrations are hand-authored and idempotent: performance indexes, and the `pg_trgm` extension with GIN trigram indexes. Trigram search was chosen over `tsvector` because PostgreSQL ships no Vietnamese or Chinese full-text configuration.
- Execution:** the operator runs migrations from the host before replacing the app container. The runtime image does not run migrations at start-up; it does not contain `drizzle-kit`.

- **Boot hook:** `instrumentation.register()` runs once per process and marks workflow runs left `running` by a crash as `resumable`; the next tick resumes them from the step journal. A database error cannot block start-up.

3 Configuration

Configuration uses environment variables only, following twelve-factor style. `.env.example` documents the operator-facing variables. The Compose file loads a host-side `.env` and overrides the service URLs that differ inside the container network. Integrations are feature-flagged by the presence of their credentials: an unset provider key simply hides that provider.

Category	Representative variables (names only)	Effect
Database	<code>DATABASE_URL</code>	PostgreSQL connection
Authentication	<code>AUTH_SECRET</code> , <code>AUTH_URL</code> , <code>AUTH_TRUST_HOST</code>	JWT signing secret and public base URL
Registration	<code>REGISTER_MODE</code> , <code>REGISTER_INVITE_CODE</code>	Sign-up is <i>open</i> , <i>invite</i> or <i>closed</i>
Local model	<code>OLLAMA_URL</code> , <code>DEFAULT_CHAT_MODEL</code> , <code>CHAT_NUM_CTX</code>	Local LLM endpoint and default model
Cloud LLM providers	<code>BYTEPLUS_*</code> , <code>CEREBRAS_*</code> , <code>DEEPSEEK_*</code> , <code>ANTHROPIC_API_KEY</code>	A key enables that provider; base URLs and retry delays are tunable
Internal model router	<code>INTERNAL_MODEL</code> , <code>INTERNAL_JSON_MODEL</code>	Pins the model used for background AI tasks (overrides the cloud-first order)
Agent loop tuning	<code>CHAT_MAX_ROUNDS</code> , <code>CHAT_REASONING_EFFORT_*</code> , <code>DIGEST_*</code> , <code>TOOL_*</code>	Runaway backstop, reasoning effort, context budgets, opt-in routing guards
Connector secrets & OAuth	<code>CONNECTOR_KEY</code> ; <code>GOOGLE_OAUTH_*</code> , <code>ATLASSIAN_OAUTH_*</code> , <code>SLACK_*</code> , <code>ZALO_*</code> , <code>TRELLO_API_KEY</code> , <code>OAUTH_PUBLIC_BASE_URL</code>	Master key for credential encryption (falls back to the auth secret when unset; see Section 5.4); each OAuth connector appears only when its client is configured
Workflow scheduler & safety	<code>WORKFLOW_TICK_SECRET</code> , <code>WORKFLOW_*_ALLOWLIST</code>	Tick authentication; per-channel recipient allowlists for scheduled outbound messages
Satellite services	<code>SEARXNG_URL</code> , <code>CONSTELLATION_TTS_URL</code> , <code>HOST_METRICS_URL</code> , <code>HOST_METRICS_TOKEN</code>	Web search, TTS, and host metrics endpoints
Monitoring & alerts	<code>LAAM_STUCK_MIN</code> , <code>LAAM_PROJECTS_DIR</code> , <code>PROACTIVE_*</code>	Stuck-session and cost-alert thresholds; local session log locations
Collector (client side)	<code>LAAM_URL</code> , <code>LAAM_MACHINE_TOKEN</code> , <code>LAAM_INTERVAL_SEC</code>	Set on each monitored workstation, not on the server

3.1 Secrets handling today

- Server secrets (`AUTH_SECRET`, `CONNECTOR_KEY`, provider API keys, OAuth client secrets and `WORKFLOW_TICK_SECRET`) live in the host's `.env` file. The file is excluded from version control. The in-network database connection string is currently set in the Compose file (Section 8, item 7). **No secret is ever sent to the browser.** LLM calls, connector calls and OAuth token exchanges all happen server-side.

- Per-user secrets (connector tokens and remote MCP server credentials) are **not** environment variables. They are stored encrypted in PostgreSQL (Section 5.4) and masked in every API response, which shows the last four characters only.
- Most variables are read when the process starts, so changing an allowlist or a secret requires an app container restart. The runbook documents this.

4 Scalability model & current constraints

The current build is designed for **exactly one application process**, and the code says so in several places. On one host this is simple and correct. It is also the main change required before production: several pieces of state live in process memory, and one boot routine assumes it is the only executor.

4.1 Single-process assumptions (from code)

Mechanism	Current implementation	Effect with more than one replica
Real-time event bus	A module-level in-memory publish/subscribe set feeds the Server-Sent Events stream: monitoring updates, workflow step progress and per-user notifications.	An event published on one replica would not reach browsers connected to another.
Login lockout & registration limiter	In-memory maps: 5 failed logins in 10 minutes lock the account for 15 minutes; registrations are limited to 10 per hour per client address, as reported by the proxy headers.	Counters would be per replica, so the effective limits would multiply.
Workflow crash recovery	At boot, every <code>running</code> run is re-flagged as <code>resumable</code> . This is correct only when the booting process is the only executor.	A second replica starting up would re-flag runs still in flight on the first. The write journal (below) prevents duplicate external writes, but runs would be disturbed.
Scheduler	No in-process cron. An external OS task sends the tick endpoint a POST every minute.	Portable across replicas, because schedule claims are atomic in SQL. The tick source still needs to become a managed job.
Caches & throttles	MCP tool discovery cache (~30 s per user) and throttles for the public geo services are in memory.	Harmless when duplicated; efficiency drops slightly.

4.2 What already scales

- **State lives in PostgreSQL.** Sessions are stateless JWTs. Conversations, tool calls, workflow runs and steps are all persisted. The app container keeps no local state apart from the items above.
- **Schedule claims and resume are concurrency-safe.** A single transaction inserts the run and advances the schedule. A unique `(schedule, slot)` key prevents double-firing. Resumable runs are claimed with `FOR UPDATE SKIP LOCKED`.
- **Idempotent external writes.** Every workflow write claims a row in a write-ahead idempotency table (`UNIQUE(run, node, iteration)`) before it sends. A completed write replays its stored output. A write

that was claimed but never recorded fails loudly and is never re-sent.

- **OAuth refresh is serialised across processes.** A PostgreSQL advisory lock guards refresh-token rotation for providers that issue single-use refresh tokens.
- **Workflow concurrency is bounded.** Each run has a budget of 200 steps and 100 foreach items. Parallel mode runs at most 2 concurrent model calls and 6 concurrent I/O calls, with a fan-out limit of 12.

■ 4.3 Target production topology PROPOSED

This topology is a proposal. None of it is deployed today. It keeps the application code as it is and moves the single-process mechanisms onto shared infrastructure.

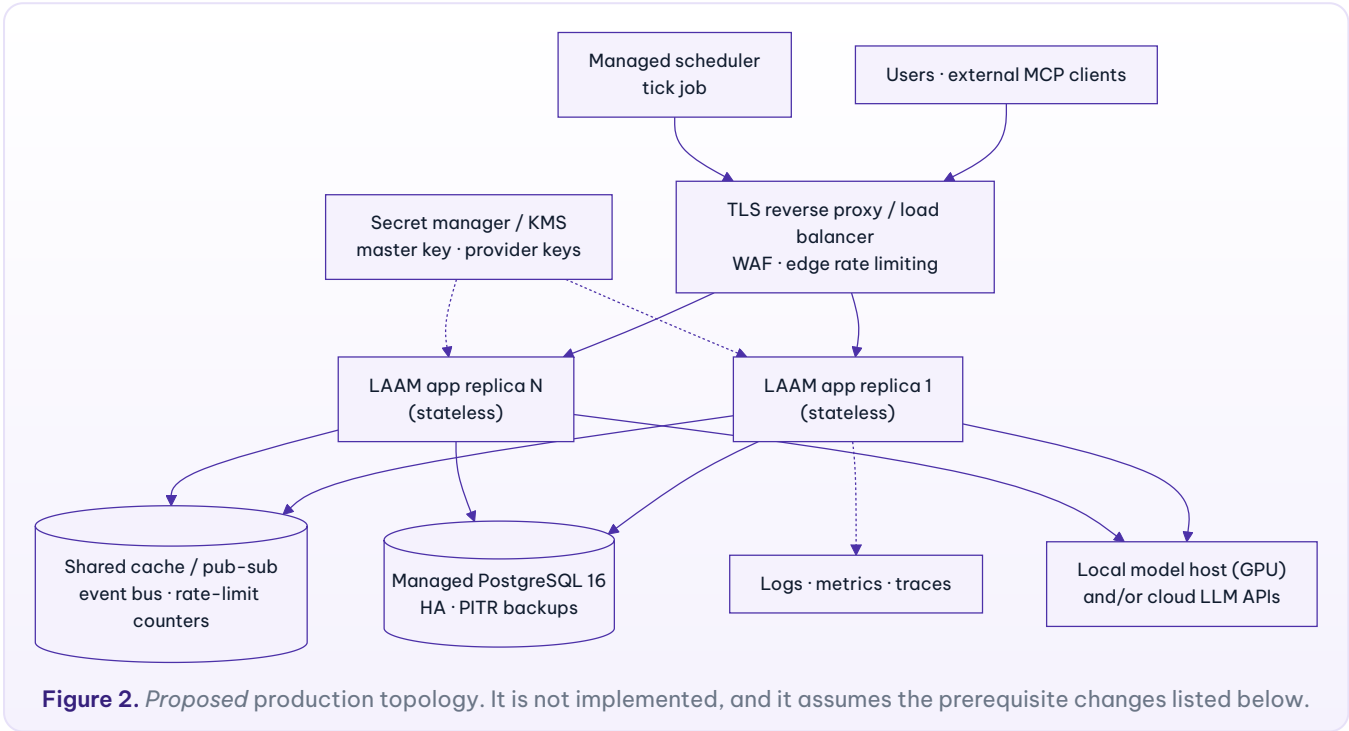


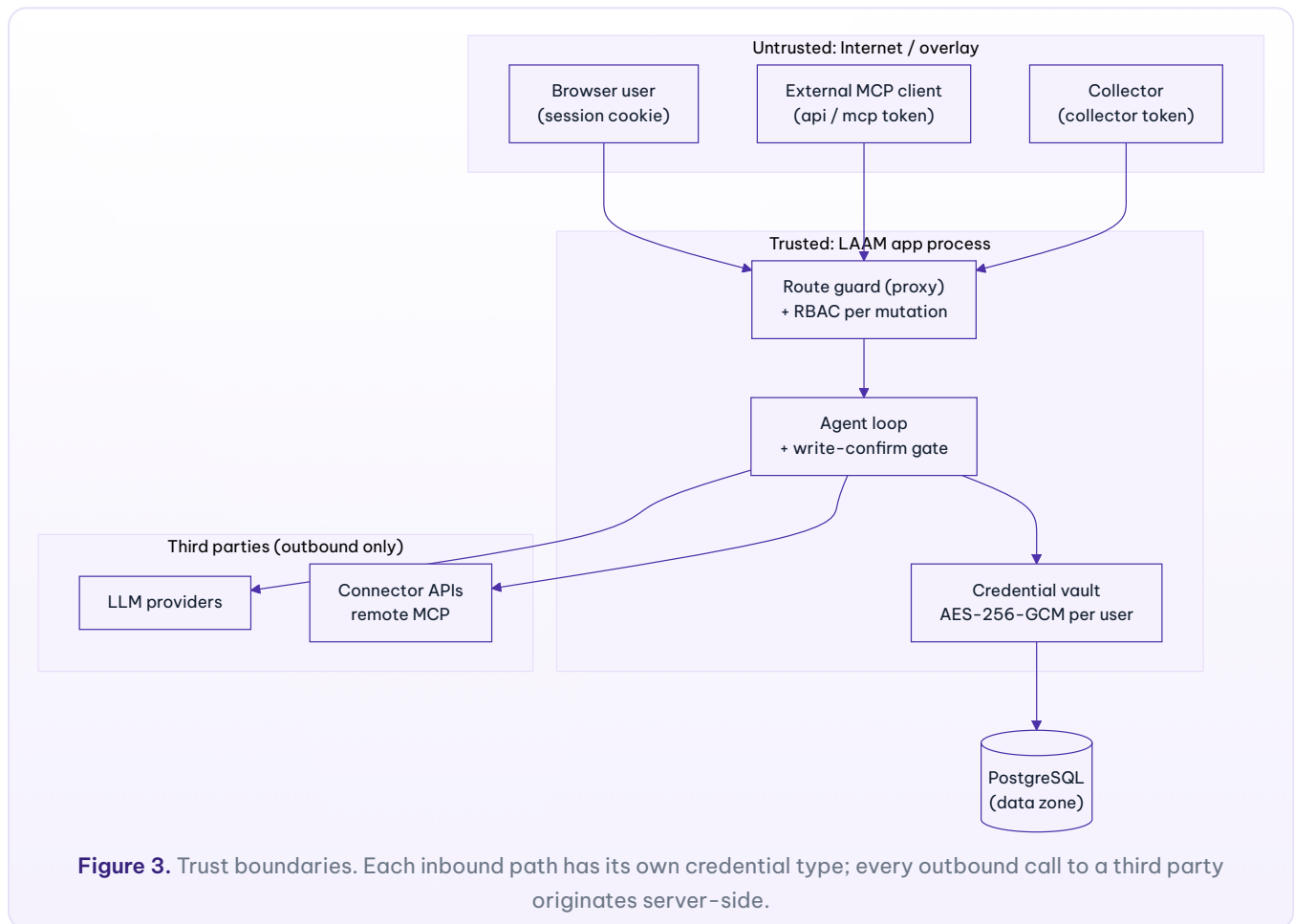
Figure 2. Proposed production topology. It is not implemented, and it assumes the prerequisite changes listed below.

Prerequisite change	Why
Move the event bus to shared pub/sub (e.g. Redis or PostgreSQL LISTEN/NOTIFY)	So that SSE updates reach users on any replica.
Move the rate limiters and lockout to a shared store, and add API-wide limits at the edge	So that the limits hold across replicas and cover every endpoint.
Replace the boot-time orphan sweep with a lease/heartbeat per run	So that one replica restarting cannot re-flag another replica's live runs.
Replace the OS tick task with a managed scheduled job	To remove the dependency on a single host.
Managed PostgreSQL with automated backups and point-in-time recovery; secrets in a secret manager or KMS	For durability, restore drills, key rotation and per-tenant key material.
Run migrations as a release job in a CI/CD pipeline	For repeatable, auditable deploys.

TO BE COMPLETED after load testing.

5 Security architecture

LAAM is a **single-organisation** application with role-based access. It is not a multi-tenant SaaS. Most of the security design goes into one question: how an AI agent that can act through a user's connected systems is kept from acting without explicit human approval in chat, and only within pre-approved limits (workflow-safe tools, allow-listed recipients) when a workflow runs unattended.



5.1 Authentication

- **Email and password** via Auth.js v5 Credentials. Passwords are hashed with **bcrypt**, and input is validated with zod.
- **JWT sessions.** On each refresh the token is **re-checked against the database**. A disabled or deleted user is signed out on their next request, and role changes apply without re-login.
- **Lockout:** 5 failures in 10 minutes locks the account for 15 minutes. Locked, unknown and disabled accounts all return the same response as a wrong password.
- **Registration** is governed by `REGISTER_MODE` (open / invite / closed), with a limit of 10 registrations per hour per client address (taken from the proxy headers).
- **Route guard:** the Next.js 16 proxy (formerly middleware) runs an edge-safe authorisation check on every non-static path. Only the landing page, login and register, the Auth.js endpoints, and three

machine endpoints are public. Each machine endpoint has its own authentication: the collector ingest, the MCP server, and the scheduler tick.

- There is **no SSO or social login** today (Section 8); LAAM's OAuth support is for *connectors* (Section 5.5).

■ 5.2 Authorization

- There are four roles: **owner**, **admin**, **member**, **viewer**. Routes that change shared or business state (chat turns, conversations, workflows, runs, schedules, connectors, MCP servers, custom agents) call `requireMutator`, so a viewer gets HTTP 403. Viewers can still edit their own profile and notifications, revoke their own tokens, and use stateless helpers (document parsing, TTS, AI workflow drafting). User and machine administration require owner or admin.
- Guards: no self role change, no demoting the last active owner, no admin-minted keys for owners/admins. Role changes, disables and token issuance go to `audit_log`.
- **Off-boarding** is a single transaction: it sets `disabled_at`, revokes all of the user's access tokens and collector credentials, and writes an audit entry. Token verification also re-checks the owner's disabled flag as defence in depth.

■ 5.3 Multi-user isolation

Each user's own data is **scoped to that user in every query**: conversations, workspaces, custom agents, workflows, runs and schedules, connector credentials, MCP server configurations and notifications. Monitoring data derived from coding-agent sessions is **organisation-shared by design** (team visibility is the point of that surface); API/MCP-created sessions are filtered by principal on the live stream.

■ 5.4 Connector credential encryption

- **Algorithm**: AES-256-GCM with a random 96-bit IV and an authentication tag, so any tampering fails decryption.
- **Per-user keys**: `HKDF-SHA256(master, salt, info = user_id)`. A current-format (v2) blob encrypted for user A **cannot be decrypted** with user B's key, so a query or code bug that crossed users would fail closed.
- **Coverage**: connector tokens, OAuth grants and remote MCP server configurations, including their auth tokens and headers. Stored blobs carry a version marker. Legacy blobs use one global key until they are re-encrypted lazily on their next write.

Honest scope

Every per-user key is derived from **one master secret** held in the server environment: `CONNECTOR_KEY`, or the session-signing secret `AUTH_SECRET` when it is unset. Making a dedicated master key mandatory is a hardening item (Section 8). The scheme gives cryptographic *user-to-user* isolation under that master. It does **not** protect against compromise of the master secret itself. The code documents per-user key material (KMS, or bring-your-own-key) as a follow-up, and the versioned blob format is the seam for adding it.

Personal access tokens (kinds `api`, `mcp` and `collector`) are high-entropy random values. Each is shown once and stored only as a SHA-256 hash. Tokens support expiry and revocation and are displayed by prefix and last four characters.

■ 5.5 OAuth for connectors

Google (Drive, Calendar, Gmail), Atlassian (Jira), Slack and Zalo connect through OAuth authorisation-code flows; Trello uses its own authorise flow. Each provider is **disabled until the operator configures**

its client credentials. A per-connector state cookie is set `httpOnly` and `SameSite=Lax`, with a short lifetime, as CSRF protection. PKCE is used where the provider supports it (Google, Zalo). Tokens are refreshed at a single execution chokepoint, under a database advisory lock. A revoked grant moves the connector to "needs reconnect" rather than failing silently.

■ 5.6 Outbound request safety (SSRF)

- **Remote MCP servers** are user-supplied URLs, fetched server-side with the user's token. The URL is checked twice. When the configuration is saved, the host is checked literally. At connect time, **DNS is resolved and every resolved IPv4 and IPv6 address is validated**, with loopback, private, link-local, cloud-metadata, IPv6 unique-local and IPv4-mapped ranges rejected.
- The **URL reader tool** and the server-side URL fetch accept only `http(s)` URLs and check the hostname against a blocklist of loopback, private and link-local names. They follow redirects manually and re-check every hop. Moving all outbound fetches onto one DNS-pinned guard is a hardening item (Section 8).
- The **catalogue connectors** (public-API and APILayer catalogues) pin each endpoint's URL template in code, so the model can only reach URLs that LAAM itself defines.
- **MCP resource limits:** at most 200 tools per server, a 15-second call timeout, a 200k-character hard cap per result with a visible truncation marker, and capped error text returned to the model.

■ 5.7 Write-confirm gating for AI actions

This is LAAM's central safety control. **In chat, the model can propose writes but cannot execute them:** every write to a connected system waits for the user's confirmation of a code-built preview, the confirmed call uses the sealed arguments, and its nonce is checked against the audit log. Every tool is classified as read or write, and any tool with an unknown kind is treated as a write (fail-closed). Remote MCP tools count as reads only when the user has marked the server as trusted *and* the tool itself declares `readOnlyHint`.

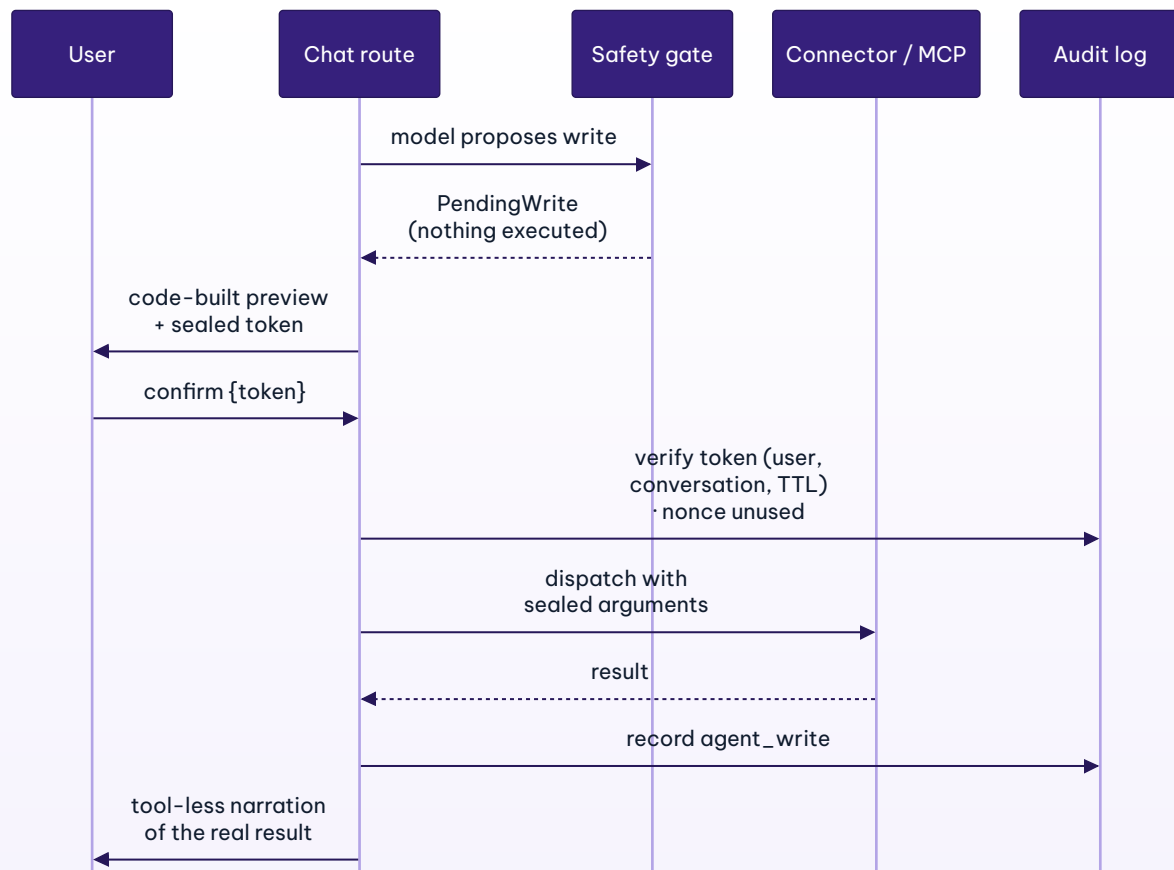


Figure 4. Write-confirm round trip in chat. The sealed token is AES-GCM encrypted, expires after 5 minutes and carries a nonce. The preview is built by code, not model prose. The arguments come from the sealed token, not from the model. The narration turn has no tools, so it cannot trigger another write.

- **Write-claim guard:** if the model claims "sent" or "created" without an executed write behind the claim, the text is replaced before it reaches the user.
- **Workflows** (scheduled or unattended) are fail-closed. They may execute a write only for tools explicitly marked workflow-safe. Outbound messages (e-mail, Slack, WhatsApp, Zalo) must go to recipients on a per-channel allowlist, and an empty allowlist blocks sending. Recipient parsing blocks e-mail header injection. MCP nodes in workflows run only tools the user trusts as reads. Dry runs mock every write.
- **AI workflow authoring** probes tools with writes blocked, never executed.

5.8 Code-execution ban

The agent cannot run code. A test in the suite scans the agent and workflow engine source and fails if process spawning, `eval`, `new Function` or the `vm` module appears. It is run with the suite; there is no CI yet (Section 8). The internal tool set and the five workflow node kinds are fixed in code, and the same test checks tool names and count. The calculator tool is a deterministic expression parser, with no `eval`.

5.9 Security headers and transport

Every response sets `Strict-Transport-Security` (180 days, including subdomains), `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, and a `Permissions-Policy` that denies the camera and limits the microphone and geolocation to the app's own

origin (both needed for voice and maps). A **Content-Security-Policy is not yet set**. It is deferred pending browser verification (Section 8). TLS is terminated at the network overlay.

■ 5.10 MCP server exposure

LAAM is also an MCP server: `POST /api/mcp`, Streamable HTTP. It exposes **9 read-only tools** over monitoring data (sessions, machines, statistics, audit log); no connector writes. Callers need a personal access token of kind `api` or `mcp`; session cookies and collector tokens are rejected. Every call is recorded as a monitored session (best-effort). Two items are **reserved rather than enforced** today: token `scopes` are stored but not checked, and apart from the audit tool, which is filtered by principal, the tools read organisation-shared monitoring data.

■ 5.11 Machine endpoints

- **Collector ingest:** a bearer collector token bound to one machine and a 5 MB request body cap enforced while streaming. The collector drops each session's transcript file path before upload; project paths and short text snippets are sent. Common credential patterns are redacted from sub-agent output before it is stored.
- **Scheduler tick:** when a secret is configured, only a constant-time match on the secret header is accepted. Without a secret, a development-only local-request check applies; a secret is required whenever the port is reachable from other machines (Section 8, item 4).

■ 5.12 Security controls summary

Control	Implementation	Status
Password authentication	bcrypt hashes, zod-validated input, Auth.js v5 Credentials	LIVE
Session management	JWT, re-validated against the DB on refresh (disable and role changes apply immediately)	LIVE
Brute-force protection	Login lockout (5 / 10 min → 15 min) and register limiter keyed on the proxy-reported client address; both in memory	LIVE
Route protection	Edge proxy check on every non-static path; explicit public allowlist	LIVE
RBAC	4 roles; <code>requireMutator</code> on state-changing business routes; owner/admin guards	LIVE
Per-user data isolation	Owner-scoped queries for chat, workflows, connectors, agents, notifications	LIVE
Off-boarding	Transactional disable, revocation of all tokens, audit entry	LIVE
Credential encryption	AES-256-GCM, HKDF per-user keys under one master secret; masked responses	LIVE
Access tokens	Random, SHA-256 hashed, shown once, expiry and revoke	LIVE
Connector OAuth	State cookie (httpOnly, Lax), PKCE where supported, locked refresh	BUILT
SSRF guard	DNS-resolved IP validation for MCP servers at connect time; hostname blocklist with per-hop redirect checks for URL reading	LIVE
AI write-confirm gate	Fail-closed classification; in chat, confirmation of a code-built preview, sealed arguments, nonce checked against the audit log	LIVE
Workflow write safety	Workflow-safe flag, recipient allowlists, WAL idempotency, dry-run mocks	LIVE

Control	Implementation	Status
Code-execution ban	Source-scan test in the suite; internal tools and node kinds fixed in code	LIVE
Security headers	HSTS, nosniff, frame deny, referrer policy, permissions policy	LIVE
MCP server	Token-authenticated, read-only tools, calls audited as sessions	BUILT
Container hardening	Non-root user, minimal Alpine runtime; SearXNG and TTS bound to loopback	LIVE
Content-Security-Policy	Deferred pending browser verification	PLANNED
API-wide / distributed rate limiting	Not implemented (auth surface only, in memory)	PLANNED
Token scope enforcement	Scopes stored, not enforced	PLANNED
SSO / enterprise identity	Not implemented	PLANNED
KMS / BYOK key material	Seam exists (versioned blobs); not implemented	PLANNED

Security testing: controls are covered by unit and integration tests in the repository's Vitest suite, which has 3,573 test cases (3,567 passing when re-run on the baseline, 2026-09-24). An independent penetration test has not yet been performed. **TO BE COMPLETED**

6 Data protection & AI data flow

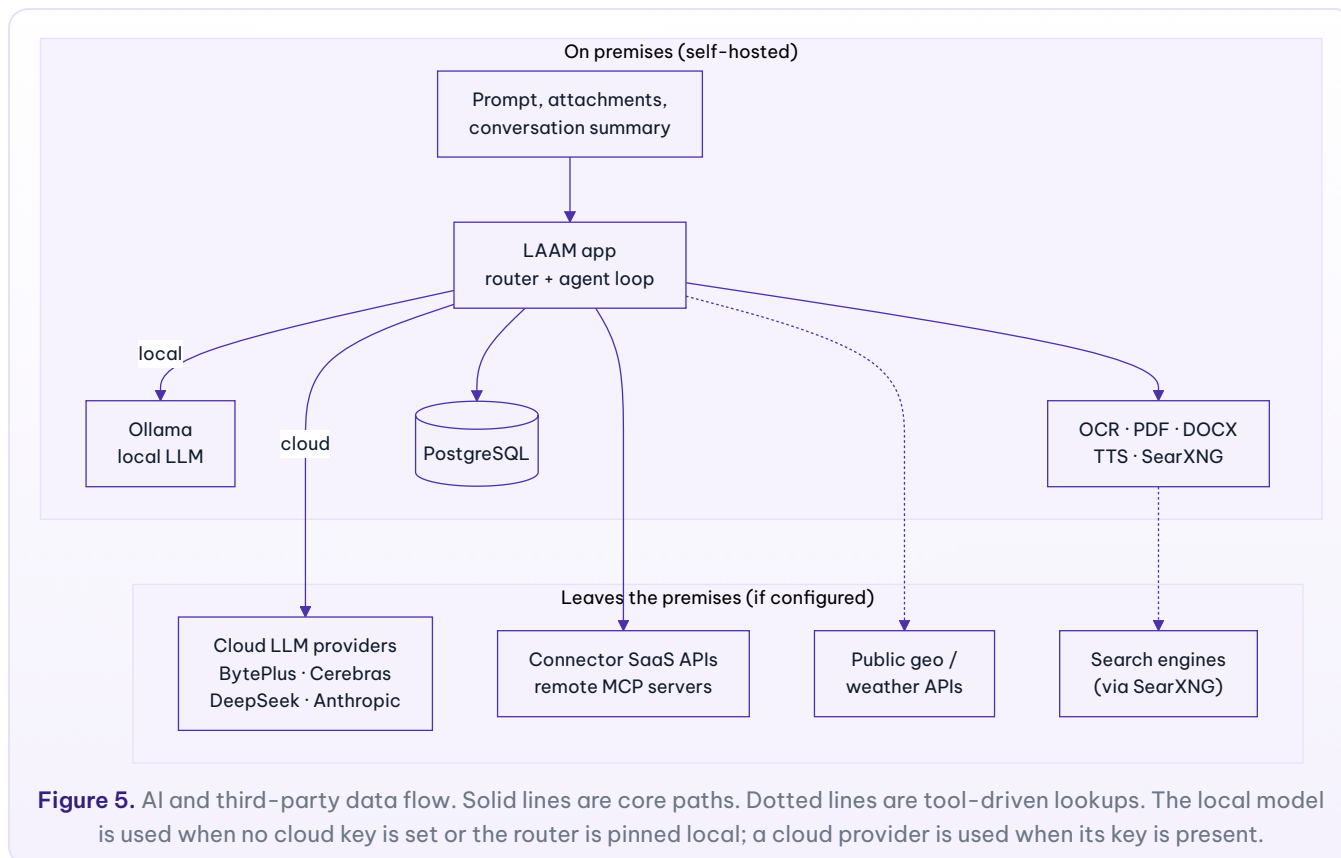
LAAM can run with every AI inference on premises, but that is **a configuration choice, not the default**. As soon as a cloud provider key is configured, background AI tasks move to that cloud provider, and the default chat model does too when the provider is BytePlus or Cerebras. This section lists every path by which data can leave the host.

6.1 What is sent to a language model

A chat turn sends the model the system prompt: persona, date, language, the tool catalogue, and instructions supplied by the user's MCP servers. It also sends the conversation history within a budget, a rolling summary of older turns, the user's message, and text extracted from attachments. When vision is used on the local model, up to two images are sent. **Tool results** go back to the model in the same loop. Large tabular results are reduced to a code-computed digest (counts, totals and sample rows) before they are sent, and the full rows stay in PostgreSQL. Common credential patterns (query-string keys, bearer tokens, GitHub tokens) are redacted from tool output before it reaches the model.

6.2 Model routing rules

- **Interactive chat:** the user picks the model. Cloud models appear only if their key is set; the preselected default is BytePlus if configured, then Cerebras, else the local model. There is **no silent cross-provider fallback**: a provider error is reported, never re-sent elsewhere.
- **Background AI tasks** (conversation summaries, workflow agent nodes, AI workflow authoring and review): `INTERNAL_MODEL` if set; otherwise the first configured cloud provider (BytePlus → Cerebras → DeepSeek → Anthropic); the local model only when no cloud key exists.



6.3 Other data leaving the host

- **Connectors and remote MCP servers:** the arguments of each tool call the user invokes or approves go to the respective SaaS or MCP endpoint.
- **Voice input:** speech-to-text uses the browser's Web Speech API, so audio is processed by the browser vendor's service. A self-hosted Whisper provider is **PLANNED** behind the existing STT interface. Speech *synthesis* is self-hosted.
- **Web search** queries go to upstream search engines through the self-hosted SearXNG. The **URL reader** fetches public pages the model selects.
- **Maps and weather:** keyless public geocoding, routing, points-of-interest and weather services receive the place names or coordinates in a request.
- **Document parsing** (PDF, DOCX, OCR) runs entirely inside the app container.

6.4 Data at rest

- All application data lives in PostgreSQL: conversations, persisted tool calls and results, workflow runs and outputs, and monitoring metadata. Connector and MCP credentials are encrypted at the application layer (Section 5.4). Encryption of the database volume depends on the host or managed service. **TO BE COMPLETED** for the target environment.
- The monitoring collector uploads parsed **session metadata** (counts, tokens, cost, tool statistics, project paths) and short text snippets (latest activity, sub-agent prompts up to 600 characters, and sub-agent outputs up to 500 characters with common credential patterns redacted). It never uploads full transcripts, and it drops the transcript file path.
- A formal data-retention policy and per-user data export or erasure workflows are not yet defined.

TO BE COMPLETED

Fully on-premises configuration

For data that must not leave the premises, the operator configures: **(1)** no cloud LLM keys, or `INTERNAL_MODEL` pinned to a local model with cloud models removed from the picker; **(2)** only on-premises connectors and MCP servers; **(3)** web search and the geo/weather tools disabled; and **(4)** voice input off until the local STT provider ships. With these settings, inference runs on the local GPU through Ollama. The trade-off is lower answer quality and speed than the cloud models give.

7 Operations

7.1 Observability

Signal	What exists today	Status
Application logs	<code>console</code> output collected as container logs (<code>docker logs</code>). Errors carry tagged prefixes, for example tool-loop failures with a conversation id. There is no structured logger yet.	LIVE
Run telemetry (in-product)	The Monitoring surface is a read model over LAAM's own chat and workflow runs. It shows the model actually used, the tools called, tokens and cost per run and per step, and live step progress over SSE.	LIVE
Tool-call audit, alerts	Every chat tool call is persisted; approved writes go to <code>audit_log</code> ; in-app alerts for stuck/costly sessions and workflow outcomes.	LIVE
AI quality	An offline evaluation harness (scenarios, graders, a tool-count scale test) writes results to a Reliability dashboard. It is run manually.	BUILT
APM / tracing / external alerting	Not implemented. <code>instrumentation.ts</code> is used only for boot-time crash recovery, not telemetry.	PLANNED

7.2 Cost tracking

Token usage is recorded per assistant message and per workflow step, together with the model that answered. Cost in USD is computed from a maintained per-model price table covering Anthropic, BytePlus, Cerebras and DeepSeek models. **Models without a known price return "unknown" rather than an estimate**, and no cost is shown when a provider did not report usage. Cost appears on the dashboard (by model, heatmap, KPIs), on each workflow run, and in proactive alerts when spending passes a threshold. Local model usage is \$0; prices are maintained by hand.

7.3 Backups and recovery

Item	State
Backup script	A <code>pg_dump</code> script with a 14-day retention sweep exists, together with a documented daily-schedule recipe and a restore procedure. It still targets the database container used before the move to the shared PostgreSQL instance , so it must be re-pointed before it can be relied on. TO BE COMPLETED

Item	State
Shared-instance backups, restore drill, RPO/RTO	Instance-level backups are managed outside the LAAM repository and not verified here; restore drills and RPO/RTO targets are not yet defined. TO BE COMPLETED
Workflow crash recovery	Built in. Runs interrupted by a restart resume from the step journal, and external writes are protected by the idempotency table (Section 4.2). LIVE

■ 7.4 Updates and migrations

1. Pull the release and build a new image locally.
2. Apply pending migrations from the host (`drizzle-kit migrate`). Migrations are journaled and safe to re-run.
3. Replace the app container (brief restart; zero-downtime deploys need the Section 4.3 changes). The database is untouched.
4. Run the post-deploy checklist: public page, auth endpoint, real browser login, local model, OCR, tick secret, and each connector's test.

■ 7.5 Health checks

- **Container health check:** every 30 seconds the image fetches the public login page. This is a **liveness** probe. It does not check database or model connectivity.
- **Dependency health** is checked in-product (OCR availability, connector and MCP "Test" actions); web search and host metrics fail soft with a clear "unavailable" state. A readiness endpoint covering database, model and satellites is **PLANNED** .

8 Hardening items before production

Planned work for a production launch, taken from the code, its comments, the changelog and an internal code review. The core security controls are in code; a few need tightening (items 6 to 10), and most of the remaining work is **operational**.

#	Item	Planned change	Area
1	Content-Security-Policy	Define and enforce a CSP after browser verification.	Web
2	API-wide rate limiting	Per-user / per-token limits on all routes; edge limits at the proxy.	Abuse
3	Shared state for replicas	Shared-store limiters and pub/sub; per-run leases.	Scale
4	Managed scheduler	Managed tick job instead of an OS task; tick secret mandatory.	Ops
5	Identity	OIDC/SAML SSO, optional MFA; enforce token scopes and per-user MCP data filtering; stable Auth.js release.	Identity
6	Key management	Dedicated master key from KMS / secret manager, mandatory at boot with no fallback and separate from the session-signing secret; rotation; re-encrypt remaining legacy blobs; BYOK via the versioned-blob seam.	Crypto
7	Secure defaults	Invite-only sign-up; app, admin UI and metrics agent bound to private interfaces; database connection string moved from the Compose file into <code>.env</code> .	Network
8	Outbound request guard	One DNS-pinned guard for all outbound fetches (URL reader, URL fetch, MCP), covering the carrier-grade NAT range and re-checked on every redirect.	Network
9	Write-confirm hardening	Atomic nonce claim and idempotent retry for confirmed MCP writes.	AI safety
10	Authentication details	Rate-limit keys taken from a trusted proxy's client address; uniform login response timing.	Identity
11	Observability	Structured logs, tracing, alerting, readiness endpoint.	Ops
12	Backups and CI/CD	Re-point backups, restore drills, RPO/RTO; CI/CD that runs the test suite (including the code-execution guard test), scanning and migrations.	Ops
13	Privacy & assurance	Self-hosted STT; third-party penetration test and AI write-gate review.	Assurance